



FLORIDA STATE UNIVERSITY
DEPARTMENT OF SCIENTIFIC COMPUTING

WRITTEN PRELIMINARY EXAMINATION
SUMMER 2024

Question - 11

Submitted By:
Mani Tyagi

Student ID :
200973108
mt22t

May 23, 2024

Contents

1	Parallel Programming[Dr. Wang]	2
1.1	Serial Code	2
1.2	Parallel Code and Load balancing	4
1.3	Scalability of Code	5

1 Parallel Programming[Dr. Wang]

1.1 Serial Code

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <chrono>
5 #include <iomanip>
6
7 using namespace std;
8
9 /**
10 * -----
11 * @brief function to generate prime numbers using the
12 *      Sieve of Eratosthenes algorithm.
13 *      https://www.topcoder.com/thrive/articles/sieve-of-eratosthenes-
14 *      algorithm#
15 * @param is_prime - array indicating whether each number is prime.
16 * @param limit     - Upper limit for generating primes.
17 * -----
18 */
19 void generate_primes(bool* is_prime, int limit) {
20
21     fill(is_prime, is_prime + limit + 1, true);
22     is_prime[0] = is_prime[1] = false;
23
24     for (int i = 2; i * i <= limit; ++i) {
25         if (is_prime[i]) {
26             for (int j = i * i; j <= limit; j += i) {
27                 is_prime[j] = false;
28             }
29         }
30     }
31 }
32
33
34 /**
35 * -----
36 * @brief Verifies Goldbach's conjecture for even numbers up to a given
37 *      limit.
38 *      Goldbach's conjecture states that every even integer greater than 2 can
39 *      be
40 *      expressed as the sum of two prime numbers.
41 * @param is_prime - Pointer to an array indicating whether each number is
42 *      prime.
43 * @param limit     - Upper limit for verifying Goldbach's conjecture.
44 * @return          - True if Goldbach's conjecture holds for all even
45 *      numbers
46 *                  up to the limit, false otherwise.
47 * -----

```

```

46  */
47  bool verify_goldbach(bool* is_prime, int limit) {
48      bool valid = true;
49
50      for (int n = 4; n <= limit; n += 2) {
51          bool found = false;
52          for (int i = 2; i <= n / 2; ++i) {
53              if (is_prime[i] && is_prime[n - i]) {
54                  found = true;
55                  break;
56              }
57          }
58          if (!found) {
59              cout << "Goldbach's conjecture is wrong for " << n << endl;
60              valid = false;
61          }
62      }
63      return valid;
64  }
65
66
67  int main() {
68
69      const int limit = 10000000;
70      bool* is_prime = new bool[limit + 1];
71
72      auto start = chrono::high_resolution_clock::now();
73      generate_primes(is_prime, limit);
74      auto end = chrono::high_resolution_clock::now();
75      cout << endl;
76
77      chrono::duration<double> diff = end - start;
78      cout << "Time to generate primes: " << diff.count() << " s" << endl;
79
80      start = chrono::high_resolution_clock::now();
81      bool result = verify_goldbach(is_prime, limit);
82      end = chrono::high_resolution_clock::now();
83      diff = end - start;
84
85      cout << "Time to verify Goldbach's conjecture: " << diff.count() << " s"
86      << endl;
87      cout << "Goldbach Conjecture: " << (result ? "Holds" : "False") << endl;
88      cout << endl;
89
90      delete[] is_prime;
91      return 0;
92  }

```

Output:

Time to generate primes: 0.0879415 s
 Time to verify Goldbach's conjecture: 0.225632 s
 Goldbach Conjecture: Holds

1.2 Parallel Code and Load balancing

The parallel code is implemented in OpenMP and for load balancing three different schedules with different chunk sizes have been tried. As shown in Table-1, the scalability is very good. The speedup is marginally better in case of Guided and Dynamic schedule in comparison to default static schedule.

```

1 bool verify_goldbach(bool* is_prime, int limit, omp_sched_t schedule_type,
2   int chunk_size) {
3
4   omp_set_schedule(schedule_type, chunk_size);
5
6   #pragma omp parallel for schedule(runtime)
7   for (int n = 4; n <= limit; n += 2) {
8       bool found = false;
9       for (int i = 2; i <= n / 2; ++i) {
10          if (is_prime[i] && is_prime[n - i]) {
11              found = true;
12              break;
13          }
14      }
15      if (!found) {
16          #pragma omp critical
17          {
18              cout << "Goldbach's conjecture is wrong for " << n << endl;
19              valid = false;
20          }
21      }
22  }
23  return valid;
24 }

```

The function `verify_goldbach` in my OpenMP code can be used with different schedule types. We can also change chunk size. A optimum chunk size can further help in reducing overhead and better load balancing.

Benefits of Adjusting Chunk Size ([ChatGpt](#)):

Load Balancing

- **Small Chunk Size:** Improves load balancing by assigning smaller portions to threads, preventing idle threads.
- **Large Chunk Size:** Reduces assignment overhead but can cause imbalance if iterations vary in execution time.

Reducing Overhead

- **Small Chunk Size:** Can increase overhead due to frequent task assignments.
- **Large Chunk Size:** Lowers task assignment overhead but may lead to idle threads with uneven workloads.

Threads	Time (s)	Goldbach Conjecture	Scalability
Static			
1	0.198131	Holds	1.00
2	0.108946	Holds	1.82
4	0.056187	Holds	3.53
8	0.0281674	Holds	7.03
16	0.0246524	Holds	8.04
28	0.0149682	Holds	13.23
Dynamic			
1	0.200183	Holds	1.00
2	0.110321	Holds	1.81
4	0.0564322	Holds	3.55
8	0.0281172	Holds	7.12
16	0.0165787	Holds	12.08
28	0.0148891	Holds	13.44
Guided			
1	0.198265	Holds	1.00
2	0.108231	Holds	1.83
4	0.0548089	Holds	3.62
8	0.027563	Holds	7.19
16	0.0167507	Holds	11.83
28	0.0146753	Holds	13.51

Table 1: Performance and Scalability of Different Schedules

1.3 Scalability of Code

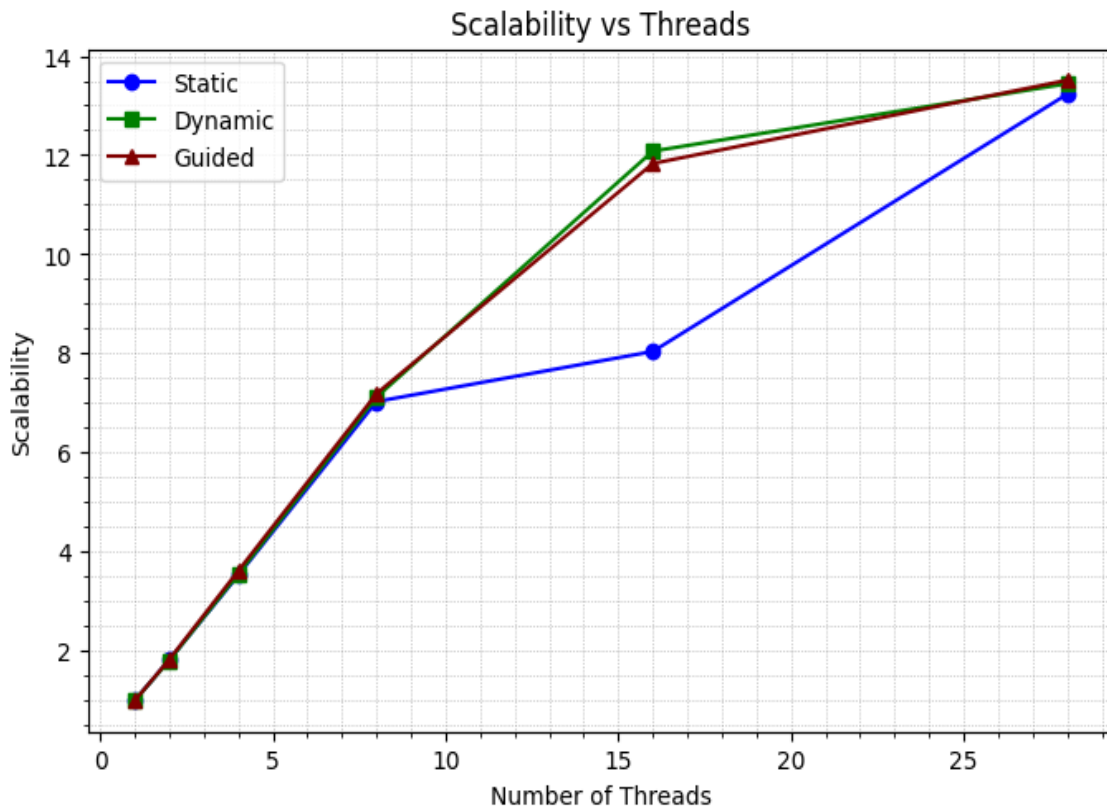


Figure 1: Scalability of code with different schedules.

NOTES:

- I have used classroom machines to run the OpenMP code. It has a maximum of 28 threads present.
- Compilation commands used:

```
-----  
[mt22t@class104 Q11-200973108]$ g++ -o openMP Q11_openMP.cpp -fopenmp  
[mt22t@class104 Q11-200973108]$ ./openMP  
-----
```

References

- <https://www.topcoder.com/thrive/articles/sieve-of-eratosthenes-algorithm#>
- <https://www.openmp.org/wp-content/uploads/SC19-FlorinaCiorba-Dynamic.pdf>
- HPC Lecture Notes, Dr. Wang